结构化剪枝

note:

- pretranined 文件夹保存了训练好的模型参数
- datasets 文件夹保存了 cifar10， cifar100 两个图像分类数据集
- 需要安装 pytorch,thop,time,copy

In [ ]:
```
import torch
import utils
from thop import profile
import time
import resnet

def get_model_info(model,val_loader,device):
    # 评估精度
    top1 = utils.get_model_accuarcy(model, val_loader, device)
    print('    Top1 Accuracy = {:.2f}%'.format(top1))
    # 评估运行成本
    input = torch.randn(1, 3, 32, 32).to(device)
    flops, params = profile(model, inputs=(input, ), verbose=False)
    print('    FLOPs = {:.4f}G'.format(flops/1000**3))
    print('    Params = {:.4f}M'.format(params/1000**2))
    # 评估推理时间
    st = time.time()
    model(input)
    et = time.time()-st
    print('    Cost Time = {:.4f}s'.format(et))
```

```
C:\Users\S.Y.F\AppData\Roaming\Python\Python39\site-packages\tqdm\auto.py:22: TqdmWarning: IProgress not found. Please update jupyter and ipyw
idgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

In [ ]:
```
# 加载数据集
dataset = 'cifar10'
num_classes,train_loader,val_loader = utils.get_dataset(dataset,'datasets/',batch_size=64)
# 加载完整模型
model = resnet.resnet34(num_classes=num_classes)
model.load_state_dict(torch.load('pretrained/cifar10_resnet34.pth', map_location='cpu')['state_dict'])
# 评估完整模型
device = 'cuda' if torch.cuda.is_available() else 'cpu'
model = model.to(device)
print('==>Info of Full Model:')
# get_model_info(model,val_loader,device)
```

```
==>Info of Full Model:
```

In [ ]:

```python
# L1-norm剪枝
import copy
def _set_module(model, submodule_key, module):
    tokens = submodule_key.split('.')
    sub_tokens = tokens[:-1]
    cur_mod = model
    for s in sub_tokens:
        cur_mod = getattr(cur_mod, s)
    setattr(cur_mod, tokens[-1], module)

rate = 0.5 #剪枝率
pruned_model = copy.deepcopy(model)

c = 0
for n,m in model.named_modules():
    # 剪枝卷积层
    if isinstance(m,torch.nn.Conv2d):
        # 第一层不修剪输入通道
        if c==0:
            mm=torch.nn.Conv2d(int(m.in_channels), int(m.out_channels*rate), kernel_size=m.kernel_size, stride=m.stride, padding=m.padding, l
            c=c+1
        else:
            mm=torch.nn.Conv2d(int(m.in_channels*rate), int(m.out_channels*rate), kernel_size=m.kernel_size, stride=m.stride, padding=m.padd

        # 计算每个filter的L1-norm
        l1_filter=torch.zeros((m.weight.data.size(0)))
        for i in range(m.weight.data.size(0)):
            l1_filter[i]=m.weight.data[i].abs().sum()
        _,l1_filter_indices = torch.sort(l1_filter, descending=True)
        mask_filter=torch.zeros((m.weight.data.size(0)), dtype=torch.bool)
        mask_filter[l1_filter_indices[0:int(m.weight.data.size(0)*rate)]]=True
        weight_filter_temp=m.weight.data[mask_filter].detach()
        weight_filter=torch.zeros((mm.out_channels,mm.in_channels,mm.kernel_size[0],mm.kernel_size[1]))
        for i in range(mm.out_channels):
            wt=weight_filter_temp[i]
            l1t=torch.zeros((wt.size(0)))
            for j in range(wt.size(0)):
                l1t[j]=wt[j].abs().sum()
            _,l1ti = torch.sort(l1t, descending=True)
            mask=torch.zeros((l1ti.size(0)), dtype=torch.bool)
            mask[l1ti[0:mm.in_channels]]=True
            mm.weight.data[i]=wt[mask]
        # 替换模块
        _set_module(pruned_model, n, mm)
    if isinstance(m,torch.nn.BatchNorm2d):
        mm=torch.nn.BatchNorm2d(int(m.num_features*rate))
```

```
        cc=0
        for i in range(int(m.num_features)):
            if mask_filter[i]!=0:
                mm.running_mean.data[cc]=m.running_mean.data[i]
                mm.running_var.data[cc]=m.running_var.data[i]
                cc=cc+1
        _set_module(pruned_model, n, mm)
    if isinstance(m,torch.nn.Linear):
        mm=torch.nn.Linear(int(m.in_features*rate),m.out_features)
        mm.weight.data=m.weight.data[:,mask]
        _set_module(pruned_model, n, mm)

pruned_model=pruned_model.to(device)
```

In [ ]:

```
# 微调模型
import torch.optim as optim
import torch.nn.functional as F

print('==>Finetruning')

epochs=160
lr=0.1
optimizer = optim.SGD(pruned_model.parameters(), lr=lr, momentum=0.9, weight_decay=1e-4)

for epoch in range(160):
    if epoch in [epochs*0.5, epochs*0.75]:
        for param_group in optimizer.param_groups:
            param_group['lr'] *= 0.1
    pruned_model.train()
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = pruned_model(data)
        loss = F.cross_entropy(output, target)
        pred = output.data.max(1, keepdim=True)[1]
        loss.backward()
        optimizer.step()
    top1 = utils.get_model_accuarcy(pruned_model, val_loader, device)
    print('   Epoch {}: Top1 Accuracy = {:.2f}%'.format(epoch,top1))
```

```
==>Finetruning
   Epoch 0: Top1 Accuracy = 35.89%
   Epoch 1: Top1 Accuracy = 43.55%
   Epoch 2: Top1 Accuracy = 51.88%
   Epoch 3: Top1 Accuracy = 58.91%
   Epoch 4: Top1 Accuracy = 65.18%
   Epoch 5: Top1 Accuracy = 68.91%
   Epoch 6: Top1 Accuracy = 70.95%
   Epoch 7: Top1 Accuracy = 76.11%
   Epoch 8: Top1 Accuracy = 77.69%
   Epoch 9: Top1 Accuracy = 77.76%
   Epoch 10: Top1 Accuracy = 79.30%
   Epoch 11: Top1 Accuracy = 79.33%
   Epoch 12: Top1 Accuracy = 80.95%
   Epoch 13: Top1 Accuracy = 81.57%
   Epoch 14: Top1 Accuracy = 80.88%
   Epoch 15: Top1 Accuracy = 82.80%
   Epoch 16: Top1 Accuracy = 83.35%
   Epoch 17: Top1 Accuracy = 83.63%
   Epoch 18: Top1 Accuracy = 81.88%
   Epoch 19: Top1 Accuracy = 83.56%
   Epoch 20: Top1 Accuracy = 82.66%
   Epoch 21: Top1 Accuracy = 84.96%
   Epoch 22: Top1 Accuracy = 83.76%
   Epoch 23: Top1 Accuracy = 84.39%
   Epoch 24: Top1 Accuracy = 85.51%
   Epoch 25: Top1 Accuracy = 84.96%
   Epoch 26: Top1 Accuracy = 85.45%
   Epoch 27: Top1 Accuracy = 85.90%
   Epoch 28: Top1 Accuracy = 84.31%
   Epoch 29: Top1 Accuracy = 85.81%
   Epoch 30: Top1 Accuracy = 85.05%
   Epoch 31: Top1 Accuracy = 86.09%
   Epoch 32: Top1 Accuracy = 85.04%
   Epoch 33: Top1 Accuracy = 86.68%
   Epoch 34: Top1 Accuracy = 84.48%
   Epoch 35: Top1 Accuracy = 86.98%
   Epoch 36: Top1 Accuracy = 86.04%
   Epoch 37: Top1 Accuracy = 86.59%
   Epoch 38: Top1 Accuracy = 85.47%
   Epoch 39: Top1 Accuracy = 86.33%
   Epoch 40: Top1 Accuracy = 86.23%
   Epoch 41: Top1 Accuracy = 85.79%
   Epoch 42: Top1 Accuracy = 85.76%
   Epoch 43: Top1 Accuracy = 85.58%
   Epoch 44: Top1 Accuracy = 85.61%
   Epoch 45: Top1 Accuracy = 86.96%
   Epoch 46: Top1 Accuracy = 87.26%
```

Epoch 47: Top1 Accuracy = 86.08%
Epoch 48: Top1 Accuracy = 87.07%
Epoch 49: Top1 Accuracy = 86.74%
Epoch 50: Top1 Accuracy = 87.25%
Epoch 51: Top1 Accuracy = 85.95%
Epoch 52: Top1 Accuracy = 86.49%
Epoch 53: Top1 Accuracy = 87.95%
Epoch 54: Top1 Accuracy = 87.75%
Epoch 55: Top1 Accuracy = 87.04%
Epoch 56: Top1 Accuracy = 86.00%
Epoch 57: Top1 Accuracy = 86.47%
Epoch 58: Top1 Accuracy = 85.85%
Epoch 59: Top1 Accuracy = 86.95%
Epoch 60: Top1 Accuracy = 87.58%
Epoch 61: Top1 Accuracy = 88.13%
Epoch 62: Top1 Accuracy = 87.30%
Epoch 63: Top1 Accuracy = 87.24%
Epoch 64: Top1 Accuracy = 87.31%
Epoch 65: Top1 Accuracy = 84.86%
Epoch 66: Top1 Accuracy = 87.27%
Epoch 67: Top1 Accuracy = 87.22%
Epoch 68: Top1 Accuracy = 87.75%
Epoch 69: Top1 Accuracy = 86.87%
Epoch 70: Top1 Accuracy = 87.20%
Epoch 71: Top1 Accuracy = 87.34%
Epoch 72: Top1 Accuracy = 87.17%
Epoch 73: Top1 Accuracy = 85.73%
Epoch 74: Top1 Accuracy = 87.87%
Epoch 75: Top1 Accuracy = 87.19%
Epoch 76: Top1 Accuracy = 86.85%
Epoch 77: Top1 Accuracy = 87.31%
Epoch 78: Top1 Accuracy = 86.72%
Epoch 79: Top1 Accuracy = 86.99%
Epoch 80: Top1 Accuracy = 90.91%
Epoch 81: Top1 Accuracy = 90.96%
Epoch 82: Top1 Accuracy = 91.42%
Epoch 83: Top1 Accuracy = 91.53%
Epoch 84: Top1 Accuracy = 91.47%
Epoch 85: Top1 Accuracy = 91.39%
Epoch 86: Top1 Accuracy = 91.48%
Epoch 87: Top1 Accuracy = 91.48%
Epoch 88: Top1 Accuracy = 91.38%
Epoch 89: Top1 Accuracy = 91.40%
Epoch 90: Top1 Accuracy = 91.40%
Epoch 91: Top1 Accuracy = 91.39%
Epoch 92: Top1 Accuracy = 91.27%
Epoch 93: Top1 Accuracy = 91.58%
Epoch 94: Top1 Accuracy = 91.08%

Epoch 95: Top1 Accuracy = 91.43%
Epoch 96: Top1 Accuracy = 91.52%
Epoch 97: Top1 Accuracy = 91.19%
Epoch 98: Top1 Accuracy = 91.28%
Epoch 99: Top1 Accuracy = 91.88%
Epoch 100: Top1 Accuracy = 91.86%
Epoch 101: Top1 Accuracy = 91.31%
Epoch 102: Top1 Accuracy = 91.65%
Epoch 103: Top1 Accuracy = 92.09%
Epoch 104: Top1 Accuracy = 91.32%
Epoch 105: Top1 Accuracy = 91.45%
Epoch 106: Top1 Accuracy = 91.45%
Epoch 107: Top1 Accuracy = 91.54%
Epoch 108: Top1 Accuracy = 91.19%
Epoch 109: Top1 Accuracy = 91.82%
Epoch 110: Top1 Accuracy = 91.56%
Epoch 111: Top1 Accuracy = 91.18%
Epoch 112: Top1 Accuracy = 91.34%
Epoch 113: Top1 Accuracy = 91.65%
Epoch 114: Top1 Accuracy = 91.31%
Epoch 115: Top1 Accuracy = 91.13%
Epoch 116: Top1 Accuracy = 91.09%
Epoch 117: Top1 Accuracy = 91.41%
Epoch 118: Top1 Accuracy = 91.04%
Epoch 119: Top1 Accuracy = 91.20%
Epoch 120: Top1 Accuracy = 91.46%
Epoch 121: Top1 Accuracy = 91.47%
Epoch 122: Top1 Accuracy = 91.45%
Epoch 123: Top1 Accuracy = 91.51%
Epoch 124: Top1 Accuracy = 91.61%
Epoch 125: Top1 Accuracy = 91.56%
Epoch 126: Top1 Accuracy = 91.66%
Epoch 127: Top1 Accuracy = 91.65%
Epoch 128: Top1 Accuracy = 91.74%
Epoch 129: Top1 Accuracy = 91.77%
Epoch 130: Top1 Accuracy = 91.60%
Epoch 131: Top1 Accuracy = 91.78%
Epoch 132: Top1 Accuracy = 91.63%
Epoch 133: Top1 Accuracy = 91.65%
Epoch 134: Top1 Accuracy = 91.72%
Epoch 135: Top1 Accuracy = 91.76%
Epoch 136: Top1 Accuracy = 91.76%
Epoch 137: Top1 Accuracy = 91.75%
Epoch 138: Top1 Accuracy = 91.76%
Epoch 139: Top1 Accuracy = 91.78%
Epoch 140: Top1 Accuracy = 91.67%
Epoch 141: Top1 Accuracy = 91.76%
Epoch 142: Top1 Accuracy = 91.65%

```
   Epoch 143: Top1 Accuracy = 91.76%
   Epoch 144: Top1 Accuracy = 91.75%
   Epoch 145: Top1 Accuracy = 91.79%
   Epoch 146: Top1 Accuracy = 91.78%
   Epoch 147: Top1 Accuracy = 91.81%
   Epoch 148: Top1 Accuracy = 91.79%
   Epoch 149: Top1 Accuracy = 91.80%
   Epoch 150: Top1 Accuracy = 91.79%
   Epoch 151: Top1 Accuracy = 91.82%
   Epoch 152: Top1 Accuracy = 91.78%
   Epoch 153: Top1 Accuracy = 91.85%
   Epoch 154: Top1 Accuracy = 91.86%
   Epoch 155: Top1 Accuracy = 91.81%
   Epoch 156: Top1 Accuracy = 91.89%
   Epoch 157: Top1 Accuracy = 91.84%
   Epoch 158: Top1 Accuracy = 91.78%
   Epoch 159: Top1 Accuracy = 91.84%
```

In [ ]:

```python
# 评估剪枝模型
print('==>Info of UnStructured Pruned Model:')
get_model_info(pruned_model,val_loader,device)
```

```
==>Info of UnStructured Pruned Model:
   Top1 Accuracy = 91.84%
   FLOPs = 0.2923G
   Params = 5.3265M
   Cost Time = 0.0110s
```